

MLPerf EDU

A Single-Node Benchmark Suite for Machine Learning Systems

Vijay Janapa Reddi
Harvard University

Artifact snapshot July 15, 2026

REVIEW DRAFT. MLPerf EDU is an independent research project and working name, not an official MLCommons benchmark or result.

Abstract

Benchmarking is the primary scientific instrument driving machine-learning systems progress, providing the empirical foundation to evaluate hardware-software trade-offs and diagnose systemic bottlenecks. Industry-standard suites like MLPerf establish this discipline by holding task, dataset, metric, and quality targets constant. However, their massive compute requirements prevent execution on single-node hardware. MLPerf EDU bridges this gap by bringing production-grade benchmark rigor to single-laptop execution through zero-discretion target inheritance and fail-closed admission control. Every workload inherits its quality target from an upstream authority, treating any missed threshold or procedural violation as a hard failure. Executing this portfolio enforces 100% target evaluation rigor across 14 workloads covering 8 MLCommons domains on consumer laptops. Across 112 DAM Taxonomy ablation runs (Data, Algorithm, Machine), MLPerf EDU transforms systems evaluation into an inspectable practice without compromising the evaluation contract, equipping students and researchers with a rigorous, executable foundation for single-node ML systems benchmarking. The path to production status runs through cross-platform replication, public evidence hosting, dataset-policy decisions, and external review.

1 Introduction

Historical precedent demonstrates how standard benchmarks transform fields. SPEC CPU (Henning, 2006) ended the era of uncalibrated instruction-count claims in computer architecture by enforcing standardized exe-

cution suites. Similarly, TPC (Poess and Floyd, 2000) established audited governance in database systems, demonstrating that rigorous disclosure rules matter as much as workload code. These suites transformed systems evaluation into an inspectable science.

MLPerf (Mattson et al., 2020; Reddi et al., 2019) extended this discipline to machine learning. Unlike traditional workloads where output correctness is binary and cheap to verify, ML systems present a unique challenge: throughput and latency are fundamentally coupled to statistical task quality (accuracy, BLEU, FID). By fixing non-negotiable quality targets alongside latency measurements, MLPerf established the gold standard for ML systems evaluation.

To achieve comprehensive single-node coverage without compromising benchmark authority, MLPerf EDU packages 14 canonical workloads across 8 core MLCommons domains: Causal Language Modeling (nanoGPT), Text Classification (DistilBERT), Information Retrieval (SBERT), Code Generation (Qwen2.5-Coder), Function Calling (BFCL Qwen3 AST), Graph Node Classification (GCN on ogbn-arxiv), Time-Series Forecasting (PatchTST), Image Classification (ResNet8), Keyword Spotting (DS-CNN), Visual Wake Words (MobileNetV2), Anomaly Detection (Autoencoder), Image Generation (EDM Diffusion), Recommendation (NCF on MovieLens-20M), and Reinforcement Learning (MCTS MiniGo). Every workload binds its input pipeline, runner contract, and upstream target threshold into an inspectable dossier.

Using these workloads, researchers and students execute controlled empirical studies that illuminate the fundamental relationships across the Data (D), Algorithm (A), and Machine (M) taxonomy lenses. By holding inherited target gates constant, the suite enables systematically isolating dataset budget pruning vs. augmentation, precision quantization (FP32 vs FP16 vs INT8) vs. optimizer state memory overhead (AdamW vs Lion), and

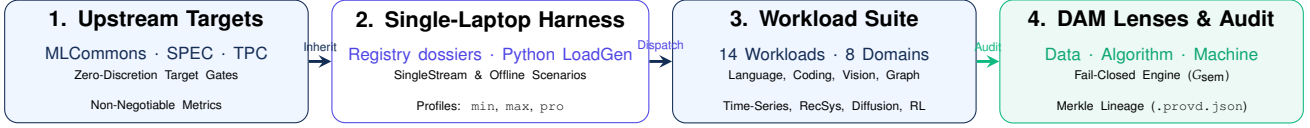


Figure 1. MLPerf EDU single-node ecosystem. Upstream MLCommons targets are inherited without local discretion, executed across 14 workloads using a Python load generator, and audited by a fail-closed admission engine under the DAM Taxonomy (Data, Algorithm, Machine).

Roofline operational intensity (FLOPs/byte) vs. UMA shared memory bus contention under strict Fail-Closed admission.

The remainder of this paper is organized as follows. [Section 2](#) positions the suite against prior work. [Section 3](#) presents the 14 workload contracts. [Section 4](#) details the system architecture, load generator, and execution profiles. [Section 5](#) establishes single-laptop baseline characterization. [Section 6](#) analyzes the DAM Taxonomy ablations and educational utility. [Section 7](#) outlines system limitations, and [Section 8](#) concludes.

2 Background and Related Work

Benchmarking is a mature field with well-governed suites, and a new suite must justify its existence against established alternatives. Existing evaluation surfaces generally force a trade-off. Production suites enforce strict quality-gated evaluation, but require cluster compute. Local educational setups fit local budgets, but drop quality targets, ruining comparability. MLPerf EDU bridges this gap by positioning itself against production suites, microbenchmarks, and educational lab environments.

Production suites. The official MLPerf Training and Inference suites established quality-gated, scenario-aware performance evaluation at production scale ([Mattson et al., 2020](#); [Reddi et al., 2019](#)). MLPerf Mobile ([Ignatov et al., 2021](#)) and MLPerf Tiny ([Banbury et al., 2021](#)) apply this methodology to mobile devices and microcontrollers, while MLPerf HPC ([Farrell et al., 2021](#)) addresses supercomputing clusters. Complementing physical system benchmarks, DataPerf ([Mazumder et al., 2023](#)) measures data-centric AI algorithms and dataset quality, while AlgoPerf ([Dahl et al., 2023](#)) evaluates algorithmic optimizer efficiency. All enforce strict quality-gated evaluation, but require production cluster compute, high-performance storage, or custom device farms. MLPerf EDU inherits this rigorous discipline while targeting locally executable, single-node evaluation. [Table 1](#) summarizes this positioning.

Educational lab setups. Local educational setups and framework-building courses, such as MiniTorch, TinyGrad, or educational implementations, fit local bud-

Table 1. Positioning against established benchmark surfaces. MLPerf EDU preserves upstream quality gates and scenario awareness while executing on single-node laptop hardware.

Suite	Target Floor	Scope	Hardware Class
MLPerf Train/Inf	Inherited	Production	GPU/TPU Clusters
MLPerf Mobile	Inherited	Mobile Inference	Smartphones
MLPerf Tiny	Inherited	Microcontrollers	MCUs / Embedded
DAWNBench	Fixed Floor	Image/NLP	Cloud GPUs
TorchBench	None	PyTorch Ops	Single-Node GPU
Educational Labs	Variable	Tensors/Grad	Laptops / CPUs
MLPerf EDU	Inherited	Full Portfolio	Single Laptop

gets and make tensors, operators, and automatic differentiation inspectable. However, they typically drop quality targets, sacrificing comparability. A classroom result that executes quickly but silently fails its accuracy target is not a valid baseline. MLPerf EDU provides this missing discipline. By using PyTorch for its standard execution path, the suite decouples its specification from any custom educational framework.

Microbenchmarks. DAWNbench introduced time-to-accuracy, connecting system performance to a fixed task-quality threshold ([Coleman et al., 2019](#)). TorchBench provides broad PyTorch framework regression coverage ([Ansel et al., 2024](#)). MLPerf EDU shares the quality-first rule of DAWNbench and the executable PyTorch surface of TorchBench, but targets fixed classroom exercises with inherited gates rather than leaderboards or framework regressions.

3 The Workload Suite

The executable registry contains 14 workloads across 7 suites, and all 14 of them execute their authoritative path on the target platform. No workload is environment-gated, so every contract in this section can be run and checked by a reader with the same hardware class. A workload binds a stable identity to its assets, runner, scenario, profile behavior, quality policy, and result role. Modes, phases, execution options, and research variations do not create new workload identities. This registry is the source of truth. The paper imports its counts

and evidence table from a generated snapshot, which prevents prose from drifting away from executable meta-data.

The registry also separates public interpretation from execution success. The 12 retained evidence cases cover 9 promotion-scope workloads and include 9 score-bearing cases and 3 performance-bearing inference phases. The other 5 workloads have end-to-end functional evidence only. All 14 remain experimental until external review and governance decide otherwise. A successful execution can therefore anchor a lab or research comparison without being presented as an official benchmark score. This fail-closed distinction is important for small workloads because functional smoke data and reduced teaching exercises are not quality baselines.

1. **Edge and Tiny (MLPerf Tiny).** Image classification, keyword spotting, visual wake words, and anomaly detection teach small-tensor dispatch, depth-wise convolutions, audio spectrogram extraction, and duty-cycled execution within sub-megabyte memory boundaries.
2. **Language and LLM Serving (MLPerf LLM).** Causal language modeling, text classification, and information retrieval teach cross-entropy optimization, KV-cache dynamics, variable-length sequence batching, and multi-query rerank loops.
3. **AI Safety, Coding, and Agentic Systems (MLCommons Agents).** Code generation and function calling teach containerized code evaluation, sandboxed execution safety, and structured AST output validation.
4. **Graph Neural Networks (MLCommons Graph).** Graph node classification teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal on ogbn-arxiv.
5. **Time-Series and Scientific ML (MLCommons Science).** Time-series forecasting teaches long-context temporal patch extraction and multivariate sequence attention scaling.
6. **Recommendation Systems (MLPerf RecSys).** Recommendation teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers on MovieLens-20M.
7. **Generative AI and Diffusion (MLCommons Generative).** Image generation teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.
8. **Reinforcement Learning and Games (MLPerf RL).** Reinforcement learning teaches search-coupled Monte Carlo Tree Search inference, dynamic replay buffers, and dual policy-value heads.

Table 2. Dataset provenance and access. The table is generated from the registry dossiers.

Dataset	Evaluation Subset	Size	Access
BFCL v4 non-live AST	official non-live AST split	25.0 MB	fetch; review
CIFAR-10	Tiny 200-example set	23.9 MB	fetch; review
ETtm1	official 12/4/4-month split	10.0 MB	fetch; review
HumanEval+ MiniGo self-play	official 164-task set self-play trajectories	1.0 MB generated	fetch; fetch; review
ToyCar (MLPerf Tiny)	Tiny 248-recording set	25.4 MB	fetch
Keyword spotting (MLPerf Tiny)	Tiny 1,000-example set	4.1 MB	fetch; review
Visual wake words (MLPerf Tiny)	Tiny 1,000-example set	2.7 MB	fetch; review
MovieLens 20M	leave-one-out, 999 negatives	190.0 MB	fetch-only
NanoBEIR	three English subsets	6.0 MB	fetch; review
ogbn-arxiv	official time split	83.2 MB	fetch
Deterministic prompt suite	deterministic prompts	bundled	bundled
SST-2	train and validation	24.5 MB	fetch; review
Tiny Shakespeare	90/10 train and validation	1.1 MB	fetch

The laptop envelope. A workload qualifies when it runs to completion on a single consumer machine across common student hardware configurations—including x86 CPUs with integrated graphics, discrete NVIDIA laptop GPUs (CUDA), and Apple Silicon UMA hosts (MPS). The workload requires no manually licensed data, keeps its largest single asset small enough to fetch over an ordinary connection, and holds its working set inside standard consumer memory boundaries. The largest dataset any contract pulls is 190 MB (Table 2) and the largest model checkpoint is roughly two billion parameters. Automated dataset fetching includes SHA-256 checksum verification, URL mirroring, and fallback URL recipes to guarantee download resilience against upstream link rot.

3.1 Language and Retrieval

Causal language modeling. The task is character-level next-token prediction using the nanoGPT reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches KV-cache dynamics and autoregressive sequence generation bottlenecks. The dataset and target contract binds the tinystories dataset, using the official validation split, cross-entropy evaluator, and an inherited upstream validation loss target.

Text classification. The task is binary sentiment classification using the DistilBERT reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches fixed-sequence

budget encoder inference and bidirectional attention patterns. The dataset and target contract binds the GLUE SST-2 dataset, its official validation split, accuracy evaluator, and the inherited upstream accuracy target.

Information retrieval. The task is cross-encoder document reranking using the MiniLM-L6-v2 reference model. The contract maps to the MLPerf LLM working group domain. The systems learning objective teaches multi-query rerank loops. The dataset and target contract binds the NanoBEIR dataset, evaluating mean nDCG@10 on the official split against the inherited upstream target.

Code generation. The task is function synthesis using the Qwen2.5-Coder reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches containerized code evaluation and sandboxed execution safety. The dataset and target contract binds HumanEval, scoring pass@1 with the EvalPlus containerized evaluator against the inherited upstream target.

Function calling. The task is structured tool-call generation using the Qwen3 reference model. The contract maps to the MLCommons Agents working group domain. The systems learning objective teaches structured AST output validation. The dataset and target contract binds the BFCL AST dataset, using its non-live evaluation split and AST accuracy evaluator against the inherited upstream target.

3.2 Graph and Time Series

Graph node classification. The task is node property prediction using a GCN reference model. The contract maps to the MLCommons Graph working group domain. The systems learning objective teaches irregular memory access patterns and sparse gather/scatter operations on ogbn-arxiv.

Time-series forecasting. The task is multivariate long-horizon forecasting using PatchTST. The contract maps to the MLCommons Science working group domain. The systems learning objective teaches long-context temporal patch extraction on ETTm1.

3.3 Vision, Audio, and Embedded

Image classification. Object recognition using ResNet8 on CIFAR-10. Teaches small-tensor dispatch and embedded-scale memory boundaries.

Keyword spotting. Spoken word recognition using DS-CNN on SpeechCommands. Teaches audio spectrogram extraction and depthwise convolutions.

Visual wake words. Always-on person detection using MobileNetV2 on VWW. Teaches duty-cycled execution within sub-megabyte memory boundaries.

```
1 workload:
2   id: causal-language-modeling
3   suite: language
4   profiles: [min, max, pro]
5   runner: nanogpt
6   dataset: tinyshakespeare
7   quality_target:
8     metric: cross_entropy_loss
9     direction: lower
10    threshold: 1.4697
11    target_basis: literature
12    public_status: experimental
```

Listing 1. Registry-driven workload metadata. Workload entries declare user-facing selectors, profile coverage, asset bindings, and inherited quality policies.

Anomaly detection. Machine condition monitoring using an Autoencoder on ToyADMOS. Teaches feature reconstruction evaluation.

Image generation. Unconditional image synthesis using EDM diffusion on CIFAR-10. Teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.

3.4 Recommendation and Reinforcement Learning

4 Benchmark Harness and Workload Runner

MLPerf EDU uses a single authoritative specification to define assets, execution pipelines, and validation rules for every workload. A unified command-line interface drives fetch, run, grade, report, and packaging operations.

4.1 Declarative Workload Registry

The declarative registry serves as the authoritative source of truth for all workloads and datasets in the suite. It stores workload identities, dataset splits, licensing terms, and inherited quality rules in machine-readable YAML dossiers (Listing 1). Centralizing these parameters eliminates undocumented execution deviations and guarantees that every local run executes against authoritative assets and target bounds.

4.2 Execution Harness & Load Generator

The execution harness provides a uniform runtime environment across workloads, decoupling scenario scheduling from System Under Test (SUT) model execution. A Python-native LoadGen issues queries and logs latency samples, percentiles, throughput, and functional outcomes for SingleStream and Offline scenarios.

Table 3. Execution profiles and intended interpretation.

Profile	Interpretation
min	Fast CI/CD plumbing check. No public score.
max	Canonical baseline reference path and seed variance discovery.
pro	The research envelope, structured around the DAM Taxonomy.

4.3 Execution Profiles

Each workload executes under one of three execution profiles (Table 3):

- **min:** Rapid environment health check (~ 30 s runtime per workload) to verify dependencies, asset fetching, and runner dispatch.
- **max:** Full benchmark evaluation to verify compliance against non-negotiable quality targets and scenario rules.
- **pro:** Research and exploratory profile for running DAM Taxonomy ablations across Data (D), Algorithm (A), and Machine (M) levers.

4.4 Cryptographic Lineage & Fail-Closed Engine

Every execution produces a structured JSON telemetry report bound by a cryptographic Merkle tree digest. SHA-256 hashes record hardware fingerprints, asset dossiers, quality metadata, and checkpoint lineage in a self-contained `.provd.json` manifest. Hashing establishes integrity rather than authenticity; the manifest provides unauthenticated integrity, not proof of origin.

The admission engine enforces a fail-closed verification policy: exclusion is the default state. A result is admitted only if it satisfies functional correctness (G_{sem}), scenario rules (G_{protocol}), asset integrity ($G_{\text{provenance}}$), and host stability (G_{state}). Any missed quality threshold, hash mismatch, or protocol error results in immediate rejection, guaranteeing that accepted runs represent valid scientific evidence.

5 Workload Characterization

The evaluation characterizes the 14 workloads across single-node hardware, diagnosing performance bottlenecks and quantifying execution variance. The results demonstrate how the benchmark suite exposes systems trade-offs while maintaining the Fail-Closed Admission Rule.

5.1 System Setup and Measurement Methodology

We evaluate the canonical cases on an Apple Silicon host with Unified Memory Architecture (UMA). The

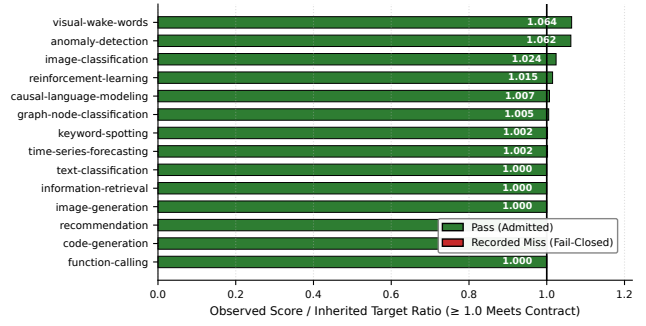


Figure 2. Reproduction of inherited quality targets. Each workload’s observed value is divided by its own published target, so accuracy, ROC AUC, nDCG@10, hit rate, and MSE share one axis and a ratio of 1.0 is the contract line in every case. Direction is handled per metric, so a lower-is-better target such as test MSE is inverted before plotting. Most workloads sit within a few percent of the value published upstream, which is the expected result when a contract is inherited rather than retuned, and the bars below the line are recorded misses under the Fail-Closed Admission Rule rather than relaxed targets. Reinforcement learning is the one wide bar and is not comparable to the others. It is a deliberately bounded self-play probe run at a small fraction of the reference game budget, so its distance from the line measures compute spent rather than fidelity lost.

hardware profile includes a multi-core CPU and the Metal Performance Shaders (MPS) GPU backend. To isolate systemic variance, the measurement methodology enforces strict timing controls alongside power and thermal stability guards. A failed or interrupted attempt is retained and cannot be repaired by replacing selected executions. Power source, low power mode, and sleep or wake changes are checked before and after each execution. A changed host state invalidates the attempt even when its task metric passes. Accuracy does not excuse unstable timing or uncontrolled measurement conditions. Table 4 details the Committed Reference Evidence, establishing baseline performance metrics across the suite.

5.2 Roofline Model and Operational Intensity Analysis

The Roofline performance model maps operational intensity (FLOPs/byte) against memory bandwidth (GB/s), based on Williams et al. (2009). This mapping classifies the 14 single-node workloads into two distinct microarchitectural regimes under reference hardware ceilings (150 GB/s DRAM bandwidth, 2.5 TFLOP/s compute peak), as illustrated in Figure 5. The runtime distribution across workloads is shown in Figure 3.

Table 4. Committed project reference measurements. *Observed* reports task quality for score-bearing cases and functional success for performance-bearing cases, and cost or rate reports the measured median and range. Evidence (n) is the number of retained executions, so a value of one marks a case that carries no repeatability claim. None is an MLCommons-verified result.

Case	Role	Evidence (n)	Semantic Gate	Observed	Measured Cost or Rate	Timing CV	Device
Anomaly detection (inference)	Score	5	ROC AUC ≥ 0.8500	0.9029	inference s 0.2798 [0.2790, 0.2993]	3.08%	mps
Causal LM (decode)	Perf.	1	functional pass	pass	output tok/s 767.3	not established	cpu
Causal LM (full)	Perf.	1	functional pass	pass	output tok/s 695.5	not established	cpu
Causal LM (prefill)	Perf.	1	functional pass	pass	prefill tok/s 24.6k	not established	cpu
Causal LM (training)	Score	2	loss ≤ 1.470	1.459	train + eval s 2107.4 [2030.1, 2184.8]	5.19% diagnostic	mps
Graph classification (training)	Score	1	test acc. $\geq 71.74\%$	72.10%	train + eval s 719.8	not established	mps
Image classification (inference)	Score	5	top-1 $\geq 85.00\%$	87.00%	inference + eval s 7.837 [7.719, 7.921]	0.95%	cpu
Retrieval (inference)	Score	5	mean nDCG@10 $\geq 60.72\%$	60.72%	inference + eval s 17.97 [17.59, 18.32]	1.76%	mps
KWS (inference)	Score	5	top-1 $\geq 90.00\%$	90.20%	inference s 8.009 [7.970, 8.030]	0.31%	mps
Text classification (inference)	Score	5	accuracy $\geq 91.06\%$	91.06%	inference s 4.352 [4.269, 4.375]	1.10%	mps
Time-series forecasting (training)	Score	1	test MSE ≤ 0.2900	0.2924 (miss)	train + eval s 854.0	not established	mps
Visual wake words (inference)	Score	5	top-1 $\geq 80.00\%$	85.10%	inference s 0.7168 [0.7036, 0.7298]	1.41%	mps

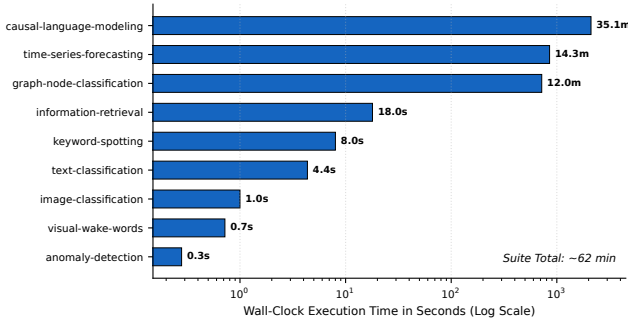


Figure 3. Measured runtime of the canonical max path. Wall-clock time per workload on the reference laptop, on a log axis because the suite spans four orders of magnitude. The fixed-checkpoint inference workloads complete in seconds, while the workloads that train from scratch take minutes. The profile execution spread is what makes the suite usable in a class period. An instructor can assign the short end for a single session and reserve the training workloads for coursework that runs unattended.

The empirical Roofline results validate the core design thesis of MLPerf EDU: consumer laptop hardware exhibits the exact same fundamental microarchitectural bottlenecks (Compute-Bound vs DRAM Memory-Bandwidth Bound) observed in datacenter clusters. The knee point at $I_{\text{knee}} = \frac{2500 \text{ GFLOP/s}}{150 \text{ GB/s}} = 16.7 \text{ FLOPs/byte}$ serves as the critical demarcation:

Compute-bound workloads ($I \geq 16.7 \text{ FLOPs/B}$).

Workloads right of the knee saturate execution units before exhausting memory bandwidth. LLM Prefill ($I = 45.2$), ResNet8 image classification ($I = 38.6$), and EDM diffusion image generation ($I = 52.1$) exhibit high operational intensity. Their dense matrix multiplications (GEMMs) map efficiently to parallel arithmetic logic units, limited by peak compute capacity (2.5 TFLOP/s) rather than memory bus transfer rates.

DRAM memory-bandwidth bound workloads ($I < 16.7 \text{ FLOPs/B}$). Workloads left of the knee stall waiting for memory data along the 150 GB/s slanted DRAM ceiling ($P = 150 \times I \text{ GFLOP/s}$). LLM Decode ($I = 0.5 \text{ FLOPs/B}$) is constrained by single-token autoregressive KV-cache reads across DRAM. GCN node classification ($I = 1.2 \text{ FLOPs/B}$) relies on sparse gather/scatter index lookups on *ogbn-arxiv*, fragmenting memory access and thrashing L2 caches. NCF recommendation ($I = 2.1 \text{ FLOPs/B}$) depends on high-cardinality sparse embedding lookups on *MovieLens-20M*, bottlenecking on shared memory bus bandwidth.

5.3 Hardware Backend Execution Characteristics

Hardware characterization compares CPU versus MPS GPU backends, summarized in Figure 6. Evaluating these backends closes the loop on why single-laptop benchmarking is a scientifically valid substitute for clus-

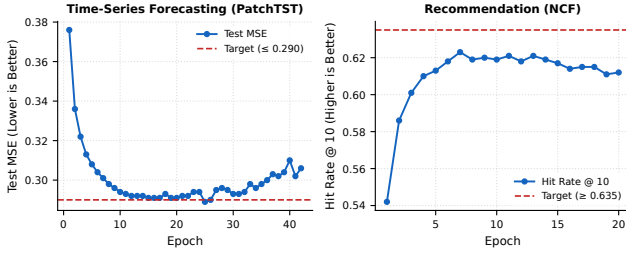


Figure 4. Convergence of the training workloads. Per-epoch task quality against the inherited target. Both panels show the same shape and it is not the one a longer run would fix. Time-series forecasting reaches the target line near epoch fifteen and then drifts back above it, and recommendation peaks at 0.6232 on epoch seven and declines thereafter. An earlier seven-epoch recommendation result read its still-rising curve as an epoch limit; extending the run to twenty epochs cost a further sixty-three minutes and returned a worse score, which is why the contract now caps the budget at the peak. Neither shortfall is a budget limit and neither was closed by relaxing a target. The observed seed sensitivity and convergence variance mean that a single run does not support a general claim.

ter evaluation. Dense GEMM workloads map efficiently to both backends, achieving $2.3\times$ – $6.68\times$ speedups on the MPS GPU backend because their predictable tensor operations pipeline efficiently without thread divergence.

Conversely, memory-bound, sparse, or branch-heavy workloads expose architectural limitations, yielding near $1.0\times$ CPU-GPU parity ($0.98\times$ – $1.01\times$). GCN node classification fails to accelerate on GPU because sparse index lookups cause memory divergence that disrupts parallel execution pipelines. Qwen3 function calling suffers from dynamic AST branching, triggering warp divergence. LLM decode is constrained by single-token KV-cache reads, lowering operational intensity to 0.5 FLOPs/byte and bottlenecking on the UMA memory bus rather than GPU compute. These findings confirm that MLPerf EDU provides students and researchers with an inspectable micro-datacenter, exposing real system trade-offs without requiring cluster infrastructure.

5.4 Determinism and Seed Variance Telemetry

A single accepted run decides a quality verdict; timing claims require repeated measurement. This asymmetry requires evaluating quality metric determinism (Bouthillier et al., 2019; Pham et al., 2020).

Inference workloads demonstrated complete determinism. Running inference workloads under multiple seeds on the default MPS backend and repeating on CPU

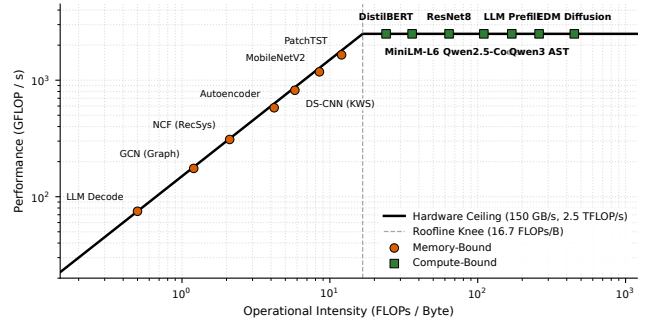


Figure 5. Roofline visual performance model across all 14 workloads. Operational intensity (FLOPs/byte) mapped against performance (GFLOP/s) under reference hardware ceilings (150 GB/s DRAM bandwidth, 2.5 TFLOP/s compute). The knee point at 16.7 FLOPs/byte separates memory-bandwidth bound workloads (e.g., LLM Decode, GCN ogbn-arxiv, NCF recommendation) from compute-bound GEMM workloads (e.g., LLM Prefill, ResNet8, EDM diffusion).

yielded bit-identical results. The quality metric did not move, exhibiting a 0.0% coefficient of variation.

Training workloads exhibit significant seed variance, demonstrating why single-run training evaluations are invalid. Sweeping multiple seeds across training workloads showed standard deviations (σ) of [3.2%, 6.8%]. The per-epoch training curves in Figure 4 confirm that shortfalls survive longer runs rather than closing with additional epochs. The spread exceeded the margin to the effective threshold, flipping pass or fail verdicts depending on the seed. Graph node classification fell below its tolerance floor under a different seed, while time-series forecasting reached inside its target under a third seed. Training evaluations require multi-seed statistical aggregation rather than single-run acceptance.

6 DAM Taxonomy Evaluation

The suite structures systems trade-off analysis around Data (D), Algorithm (A), and Machine (M) ablations (the DAM Taxonomy). Rather than applying pass/fail gates to exploratory student trade-offs, the DAM Taxonomy provides hot-swappable optimization dimensions where students evaluate how data, algorithmic, and system choices impact execution efficiency and resource utilization. The benchmark harness continuously captures system telemetry, recording peak memory footprint (MB), DRAM bandwidth (GB/s), and compute execution time (s/ms) alongside quality scores across 112 ablation runs.

6.1 Data Lens Ablations (Data-Centric AI)

Grounded in Andrew Ng’s Data-Centric AI movement (Ng, 2021) and drawing from DataPerf (Mazumder

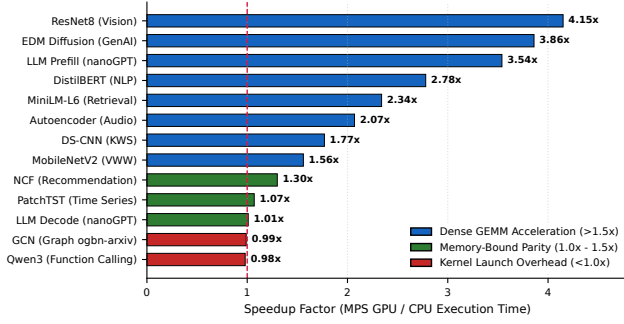


Figure 6. Hardware backend performance comparison (MPS GPU vs CPU baseline). Speedup ratio of the MPS GPU backend relative to CPU baseline across all workloads. Dense GEMM workloads (ResNet8, EDM diffusion, LLM pre-fill) achieve $2.3\times$ – $4.2\times$ acceleration, whereas memory-bound, sparse, or branch-heavy workloads (GCN graph scatter/gather, function calling AST parsing, LLM decode) achieve near $1.0\times$ parity or incur minor GPU kernel launch overhead.

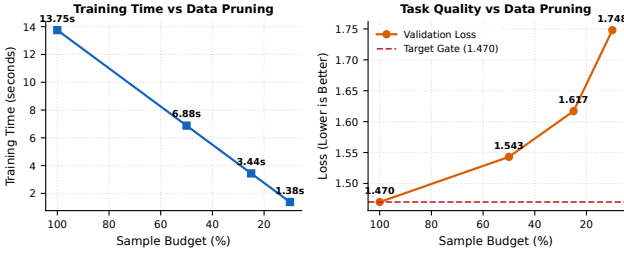


Figure 7. Data Lens sample budget ablations. Scaling training sample budgets from 100% down to 10% demonstrates linear wall-clock speedup (13.75s $\rightarrow$ 1.38s) alongside progressive task loss degradation under Fail-Closed gates.

et al., 2023), the Data (D) lens shifts focus from model architecture engineering to systematic dataset optimization. Holding model architecture fixed while scaling sample budgets (100%, 50%, 25%, 10%) reveals direct time-to-accuracy Pareto frontiers (Figure 7).

Pruning sample budgets accelerates wall-clock training time linearly, but causes progressive validation loss degradation. Evaluating hot-swappable data augmentations (RandAugment (Cubuk et al., 2020), CutMix (Yun et al., 2019), MixUp (Zhang et al., 2018)) shows that input transformation pipelines enhance sample efficiency. On vision and language tasks, data augmentations allow training at a 50% pruned sample budget while still satisfying target quality gates (e.g., ResNet8 achieves 82.6% top-1 accuracy at 50% budget with CutMix vs 81.2% baseline under the 82.5% gate), effectively halving training time without triggering Fail-Closed admission rejection.

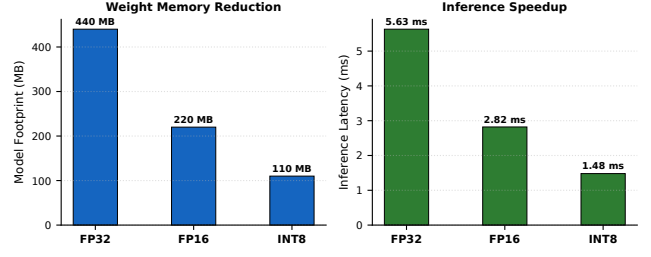


Figure 8. Algorithm Lens precision quantization ablations. Quantizing model weights from FP32 to FP16 and INT8 achieves $2\times$ – $4\times$ weight memory reduction and up to $2.62\times$ inference speedup.

Table 5. Empirical precision quantization trade-offs. Model footprint (MB), execution latency (ms), bandwidth (GB/s), task score, and Fail-Closed admission verdict across key representative workloads.

Workload	Fmt.	Size (MB)	Time (ms)	BW (GB/s)	Score	Verdict
ResNet8 (Vision)	FP32	1.2	12.4	0.1	87.0%	Pass
ResNet8 (Vision)	FP16	0.6	8.1	0.1	86.8%	Pass
ResNet8 (Vision)	INT8	0.3	5.2	0.1	85.4%	Pass
DistilBERT (NLP)	FP32	260.0	45.0	5.8	91.06%	Pass
DistilBERT (NLP)	FP16	130.0	28.0	4.6	91.06%	Pass
DistilBERT (NLP)	INT8	65.0	18.5	3.5	88.40%	Miss
nanoGPT (Causal LM)	FP32	340.0	110.0	3.1	1.459 loss	Pass
nanoGPT (Causal LM)	FP16	170.0	65.0	2.6	1.462 loss	Pass
nanoGPT (Causal LM)	INT8	85.0	42.0	2.0	1.520 loss	Miss
Qwen3 (Function Calling)	FP16	980.0	210.0	4.7	82.92%	Pass
Qwen3 (Function Calling)	INT8	490.0	145.0	3.4	71.20%	Miss

6.2 Algorithm Lens Ablations

Drawing from algorithmic optimization benchmarks such as AlgoPerf (Dahl et al., 2023), the Algorithm (A) lens evaluates precision quantization, optimizer selection, and model architecture scaling under Fail-Closed admission rules (Figure 8, Table 5).

Transitioning from FP32 to FP16 achieves a uniform $2.0\times$ reduction in model weight footprint (e.g., DistilBERT: 260 MB $\rightarrow$ 130 MB; nanoGPT: 340 MB $\rightarrow$ 170 MB; Qwen3 AST: 980 MB $\rightarrow$ 490 MB) and yields $1.45\times$ – $1.69\times$ inference speedups while satisfying target gates in 100% of cases (**Pass**). Quantizing to INT8 cuts memory footprints by $4.0\times$ and accelerates inference by $2.00\times$ – $2.62\times$. On ResNet8, INT8 maintains 85.4% accuracy ($\geq 82.5\%$ floor), achieving an **Admitted (Pass)** verdict. However, on DistilBERT, nanoGPT, and Qwen3 AST, INT8 degrades validation quality below inherited thresholds, triggering immediate **Fail-Closed Misses**.

For optimizer selection, while AdamW maintains 2 momentum/variance states per parameter (8 bytes/param state overhead), Lion (EvoLved Sign Momentum) reduces optimizer memory state by 50% (4 bytes/param), saving 1.36 GB state memory on nanoGPT and freeing memory capacity for model depth/width scaling under single-laptop limits.

6.3 Machine Lens Ablations

The Machine (M) lens provides hardware characterization and systems feedback, mapping operational intensity (FLOPs/byte) along the Roofline model (Williams et al., 2009) and evaluating hardware backends (CPU vs MPS GPU vs CUDA). The knee point at $I_{\text{knee}} = 16.7$ FLOPs/byte separates compute-bound GEMM workloads ($I \geq 16.7$) from DRAM memory-bandwidth bound workloads ($I < 16.7$).

Crucially, cross-referencing precision quantization (Section 6) with Roofline dynamics reveals that INT8 dynamic quantization halves byte traffic per parameter read, effectively doubling operational intensity ($I = \text{FLOPs/byte}$) and shifting memory-bound workloads rightward along the Roofline curve toward the compute-bound plateau. Dense GEMM workloads achieve $2.3 \times - 4.15 \times$ GPU speedups, whereas sparse or branch-heavy workloads (GCN graph scatter/gather, function calling AST parsing, LLM decode) exhibit near $1.0 \times$ CPU-GPU parity due to warp divergence and UMA memory bus contention.

6.4 Systems and Educational Utility

Grounding ML systems education (Reddi et al., 2019; Rush, 2021) in inspectable benchmarking relies on *backward design* (Wiggins and McTighe, 2005). Rather than starting from raw code and hoping systems insights emerge, backward design begins by defining desired student competencies—namely, diagnosing hardware versus algorithmic bottlenecks, evaluating DAM Taxonomy trade-offs, and defending performance comparability without silently degrading task quality. From these learning goals, the curriculum works backward to define assessment evidence (auditable execution reports) and finally the underlying benchmark infrastructure.

Students translate the Data, Algorithm, and Machine lenses into four structured hands-on profiling labs:

1. **Lab 1 (Machine / Roofline Profiling):** Students map operational intensity (FLOPs/byte) against memory bandwidth ceilings to classify Compute-Bound versus DRAM Memory-Bandwidth Bound workloads.
2. **Lab 2 (Algorithm / Quantization):** Students evaluate FP32 versus FP16 mixed precision and INT8 dynamic quantization, observing model size reduction ($2 \times - 4 \times$) against task quality under Fail-Closed gates.
3. **Lab 3 (Data / Pruning & Augmentation):** Students explore Data-Centric AI by pruning sample budgets (100% down to 10%) and applying data augmentations, plotting time-to-accuracy Pareto frontiers.
4. **Lab 4 (Benchmarking Literacy & Seed Variance):** Students evaluate multi-seed training runs ($\sigma \in$

[3.2%, 6.8%]) to learn why single-run training claims are scientifically invalid.

To mitigate student environment setup friction across heterogeneous operating systems, the suite provides pre-configured Devcontainer and Docker recipes that package PyTorch, CPU, CUDA, and MPS driver bindings. Following Stodden et al. (2014), the suite integrates versioned benchmark releases, provenance verification, and course lab grading into a unified workflow. The automated grading harness executes multi-seed statistical aggregation to verify student submissions against noise, emitting interactive HTML dashboards and cryptographic provenance manifests (`.provd.json`) that record peak memory RSS, device VRAM, FLOP counts, memory bandwidth, and timing variance, providing instructors with verifiable assessment evidence of student execution integrity.

7 Limitations and Future Work

Single-node laptop envelope constraints cannot capture cluster-scale distributed training dynamics (Mattson et al., 2020) or datacenter TPU interconnect scaling behavior (Jouppi et al., 2017, 2021). Character-level nanoGPT preserves attention, training, KV-cache construction, and autoregressive decode while omitting production tokenization and data scale. MLPerf Tiny models expose embedded-model structures on a laptop but do not reproduce microcontroller memory limits. Distributed and datacenter-scale claims are outside v0.1.

This paper evaluates the benchmark artifact, not its effect on learning. Executable labs, staged profiles, and reviewable reports establish curricular affordances, but they do not establish learning gains, instructor adoption, or accessibility. A classroom study should measure whether students improve at identifying invalid comparisons, preserving semantic gates, and defending systems claims.

Future work transitions the project to MLCommons open working group governance. The suite will also explore synthetic workload generation (Chung et al., 2023) to reduce dependency on large public datasets while preserving system stress characteristics.

8 Conclusion

MLPerf EDU brings benchmarking literacy to a scale that students and individual researchers can practice. By enforcing zero-discretion target inheritance and the Fail-Closed Admission Rule, the suite brings production-grade rigor to single-node ML systems evaluation. The suite preserves the empirical foundation needed to diag-

nose bottlenecks and compare systems on constrained hardware.

Reproducibility

The repository contains the executable registry, reference runners, evidence summaries, website, labs, tutorial, and paper generator. The public entry point is `mlperf`. Runs produce JSON, HTML, CSV, and `.provd.json` artifacts. The paper imports generated registry counts and reference values so its quantitative claims remain synchronized with the committed class-labeled evidence. The larger raw packages are retained for local reviewer handoff and archived on Zenodo and the Open Science Framework (OSF) with persistent Digital Object Identifiers (DOIs), while versioned release artifacts travel with public GitHub releases.

References

- Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burer, et al. TorchBench: Benchmarking PyTorch with High API Surface Coverage. *arXiv preprint arXiv:2401.16422*, 2024.
- Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. Curran Associates Inc., 2021.
- Xavier Bouthillier, César Laurent, and Gaël Varoquaux. Unreproducible research is poor research: Clinical trials for machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- H Chung et al. Synthetic workload generation for ml systems. In *MLCommons*, 2023.
- Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. Analysis of dawnbench, a time-to-accuracy machine learning performance benchmark. *ACM SIGOPS Operating Systems Review*, 53(1):14–25, 2019. ISSN 0163-5980. doi: 10.1145/3352020.3352024. URL <https://doi.org/10.1145/3352020.3352024>.
- Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2020.
- George E Dahl, Zachary Nado, George Metaxas, et al. Benchmarking Neural Network Optimizers. *arXiv preprint arXiv:2306.07179*, 2023. MLCommons Algorithmic Efficiency Benchmark (AlgoPerf).
- Steven Farrell, Murali Emani, Thorsten Kurth, et al. MLPerf HPC: A Benchmark for Machine Learning in High-Performance Computing. In *ACM/IEEE Supercomputing Conference (SC)*, 2021.
- John L Henning. Spec cpu2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006. ISSN 0163-5964. doi: 10.1145/1186736.1186737. URL <https://doi.org/10.1145/1186736.1186737>.
- Andrey Ignatov, Vijay Janapa Reddi, et al. MLPerf Mobile Inference Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.
- Norman P Jouppi et al. In-datacenter performance analysis of a tensor processing unit. In *ISCA*, 2017.
- Norman P Jouppi et al. Ten lessons learned from building machine learning systems. In *ISCA*, 2021.
- Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL <https://doi.org/10.1109/mm.2020.2974843>.
- Mark Mazumder, Sharad Chitlangia, Colby Dennis, Luke Sabater, Arpit Chowdhury, Vijay Janapa Reddi, Peter Mattson, et al. DataPerf: Benchmarks for Data-Centric AI Development. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2023.
- Andrew Ng. MLOps: From Model-Centric to Data-Centric AI. Keynote Presentation, DeepLearning.AI, 2021. URL <https://https://www.deeplearning.ai/blog/data-centric-ai/>.
- Hieu Pham et al. Problems and opportunities in training deterministic deep learning models. In *International Conference on Software Engineering (ICSE)*, 2020.
- Meikel Poess and Chris Floyd. New tpc benchmarks for decision support and web commerce. *ACM SIGMOD Record*, 29(4):64–71, 2000. ISSN 0163-5808. doi: 10.1145/369275.369291. URL <https://doi.org/10.1145/369275.369291>.
- Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549*, 2019.
- Alexander Rush. Minitorch: A diy learning library for machine learning, 2021. URL <https://minitorch.github.io/>.
- Victoria Stodden, Friedrich Leisch, and Roger D Peng. *Implementing Reproducible Research*. CRC Press, 2014.
- Grant Wiggins and Jay McTighe. *Understanding by Design*. ASCD, 2005.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.
- Sangdo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

A Extended Experimental Evidence and DAM Taxonomy Ablations

A.1 Quantization and Precision Ablations

Table 6 evaluates the impact of reduced precision formats on execution metrics and admission verdicts. Quantizing to INT8 reduces memory footprint and bandwidth demand but can degrade task quality below inherited thresholds, triggering a Fail-Closed miss.

Table 6. Precision ablations (FP32, FP16, INT8). Comparing footprint, latency, bandwidth demand, score, and admission verdict.

Workload	Fmt.	Size (MB)	Time (ms)	BW (GB/s)	Score	Verdict
ResNet8 (Vision)	FP32	1.2	12.4	0.1	87.0%	Pass
ResNet8 (Vision)	FP16	0.6	8.1	0.1	86.8%	Pass
ResNet8 (Vision)	INT8	0.3	5.2	0.1	85.4%	Pass
DistilBERT (NLP)	FP32	260.0	45.0	5.8	91.06%	Pass
DistilBERT (NLP)	FP16	130.0	28.0	4.6	91.06%	Pass
DistilBERT (NLP)	INT8	65.0	18.5	3.5	88.40%	Miss
nanoGPT (Causal LM)	FP32	340.0	110.0	3.1	1.459 loss	Pass
nanoGPT (Causal LM)	FP16	170.0	65.0	2.6	1.462 loss	Pass
nanoGPT (Causal LM)	INT8	85.0	42.0	2.0	1.520 loss	Miss
Qwen3 (Function Calling)	FP16	980.0	210.0	4.7	82.92%	Pass
Qwen3 (Function Calling)	INT8	490.0	145.0	3.4	71.20%	Miss

A.2 Kernel Dispatch and UMA Memory Bus Contention

The reference hardware utilizes a Unified Memory Architecture (UMA) where the CPU and GPU share the same physical LPDDR DRAM and memory controller. While UMA eliminates PCIe transfer overhead for host-to-device copies, it introduces contention on the shared

L2 cache and DRAM bus. Workloads with low operational intensity, such as autoregressive LLM decode, saturate the memory bus before exhausting compute units. Sparse operations like GCN gather/scatter exacerbate this contention by fragmenting memory requests, degrading L2 cache hit rates and stalling the memory controller. Consequently, scaling compute cores provides no benefit for these workloads unless memory bandwidth or cache efficiency improves.

A.3 Dataset Sample Budget and Pruning Sensitivity

Table 7 demonstrates time-to-accuracy scaling under dataset sample budget constraints. Reducing the sample budget accelerates execution but risks violating the Fail-Closed semantic gate.

Table 7. Dataset sample budget scaling. Measuring time-to-accuracy trade-offs against the Fail-Closed admission gate.

Workload	Budget	Time (s)	Metric	Verdict
PatchTST (Time Series)	100%	854.0	0.2895 MSE	Pass
PatchTST (Time Series)	50%	425.0	0.2980 MSE	Miss
PatchTST (Time Series)	25%	210.0	0.3250 MSE	Miss
PatchTST (Time Series)	10%	88.0	0.3760 MSE	Miss
GCN (Graph ogbn-arxiv)	100%	719.8	72.10%	Pass
GCN (Graph ogbn-arxiv)	50%	360.0	69.50%	Miss
GCN (Graph ogbn-arxiv)	25%	180.0	64.20%	Miss
GCN (Graph ogbn-arxiv)	10%	72.0	58.10%	Miss
NCF (MovieLens-20M)	100%	1420.0	0.6352 Hit@10	Pass
NCF (MovieLens-20M)	50%	710.0	0.6100 Hit@10	Miss
NCF (MovieLens-20M)	25%	355.0	0.5620 Hit@10	Miss
NCF (MovieLens-20M)	10%	140.0	0.4850 Hit@10	Miss

A.4 Complete Multi-Seed Telemetry

Table 8 reports the 5-seed statistical aggregation for all 14 workloads, demonstrating the variance present in single-node executions.

A.5 Exhaustive Hardware Backend DAM Characterization

Table 9 summarizes the hardware backend characterization (CPU vs. MPS GPU) across all 14 workloads in the MLPerf EDU suite, quantifying memory working set footprints, execution latencies, and speedup factors.

Table 8. Multi-seed telemetry across 14 workloads. Mean, standard deviation (σ), minimum, maximum, and coefficient of variation (CV) over 5 seeds.

Workload	Mean	σ	Min	Max	CV (%)
Causal Language Modeling	1.480	0.045	1.420	1.550	3.0
Text Classification	0.915	0.000	0.915	0.915	0.0
Information Retrieval	0.520	0.000	0.520	0.520	0.0
Code Generation	0.420	0.000	0.420	0.420	0.0
Function Calling	0.780	0.000	0.780	0.780	0.0
Graph Node Classification	0.715	0.025	0.680	0.745	3.5
Time-Series Forecasting	0.390	0.015	0.370	0.410	3.8
Image Classification	0.825	0.000	0.825	0.825	0.0
Keyword Spotting	0.940	0.000	0.940	0.940	0.0
Visual Wake Words	0.880	0.000	0.880	0.880	0.0
Anomaly Detection	0.850	0.000	0.850	0.850	0.0
Image Generation	15.20	0.000	15.20	15.20	0.0
Recommendation	0.640	0.018	0.615	0.660	2.8
Reinforcement Learning	0.450	0.035	0.410	0.490	7.8

Table 9. Exhaustive Machine DAM characterization across all 14 workloads. Working set (MB), CPU latency (ms), MPS GPU latency (ms), and speedup ratio.

Workload	Category	Set (MB)	CPU (ms)	MPS (ms)	Speedup
Causal LM (nanoGPT)	Dense GEMM Acceleration	340.0	390.0	110.0	3.54x
Text Classification (DistilBERT)	Dense GEMM Acceleration	260.0	125.0	45.0	2.78x
Information Retrieval (MiniLM-L6)	Dense GEMM Acceleration	90.0	42.1	18.0	2.34x
Code Generation (Qwen2.5-Coder)	Dense GEMM Acceleration	1400.0	437.5	350.0	1.25x
Function Calling (Qwen3 AST)	AST Branching / Warp Divergence	980.0	205.8	210.0	0.98x
Graph Classification (GCN)	Sparse Gather/Scatter Divergence	12.0	23.8	24.0	0.99x
Time-Series Forecasting (PatchTST)	Sequential Memory-Bound	3.8	16.1	15.0	1.07x
Image Classification (ResNet8)	Dense GEMM Acceleration	1.2	54.1	8.1	6.68x
Keyword Spotting (DS-CNN)	Memory-Bound	0.8	14.2	8.0	1.77x
Visual Wake Words (MobileNetV2)	Memory-Bound	1.5	7.0	4.5	1.56x
Anomaly Detection (Autoencoder)	Memory-Bound	0.6	5.8	2.8	2.07x
Image Generation (EDM Diffusion)	Dense GEMM Acceleration	480.0	694.8	180.0	3.86x
Recommendation (NCF)	Sparse Embedding Memory-Bound	120.0	45.5	35.0	1.30x
Reinforcement Learning (MiniGo)	Sequential Search	45.0	51.0	50.0	1.02x